Campaign POC — One Table, End To End, On Azure

Date: 2026-08-01 Scope: **Campaign only**. Prove the whole pipeline on the smallest object before touching the other 8. Companion doc: azure_migration_plan.md — the full multi-service plan and trade-offs. **This POC implements that plan for one object;** every Azure tool used here is described and justified in the plan (see its §0.6 and §4).

Why Campaign

- Smallest object: ~41K rows (~50 MB). Cheap and fast to run many times.
- Already the most developed: raw table, dedup view, materialized latest, and 19 DQ rules exist.
- If the full flow (ingest -> stage -> check -> clean -> ready to push back) works here, it repeats for every other object by changing names only.

The Invariant This POC Must Prove

> **Raw may contain duplicates. Staging must not. Final (clean, push-ready) must not start until it is safe.**

Enforced as three hard gates:

1. **Raw gate:** append-only. Duplicates and history are allowed and expected.
2. **Staging gate:** exactly one row per Campaign `Id` (latest by `SystemModstamp`), deleted rows excluded.
3. **Final gate:** the clean / push-ready set is only built when **0 CRITICAL DQ failures** remain for the rows in scope. If any critical failure exists, final does **not** start.

And the whole thing is **incremental** and **restartable**: at any point we can ask the control tables "where are we?" and re-derive state from the start without guessing.

Part A — What Is Currently Built For Campaign

Stage	Object today	Notes
Ingest	`raw.salesforce_campaign`	Bulk API 2.0 via `python.py`; ~41K rows incl. 529 duplicates; all `NVARCHAR(MAX)`
Control	`ctl.etl_run_control`, `ctl.watermark_control`, `ctl.object_registry`, `ctl.loaded_salesforce_campaign_ids`	run log, watermark cursor, metadata, resume ids
Staging (view)	`staging.vw_campaign_latest`	`ROW_NUMBER() PARTITION BY Id ORDER BY SystemModstamp DESC`, excludes `IsDeleted`
Staging (table)	`staging.campaign_latest`	materialized 40,775 unique rows + `staging_is_duplicate`, `staging_duplicate_count`, `staging_created_at`
Checks	19 rules CAM-001..CAM-017 + CAM-URL-001	see `mohey_work/Tables/campaign/campaign_staging_dq_PROD.sql`
Checks (framework)	`dq.dq_rule_catalog` + watermark-incremental runner	reruns only rows past the DQ cursor
Clean	not built yet	this POC adds it
Push-ready / Writeback	designed only	this POC adds a staged, approval-gated set

Verified evidence (2026-07-29 run): raw non-deleted 41,304 -> staging 40,775 (529 duplicates removed); 0 CRITICAL failures, 9 HIGH, ~4,901 MEDIUM, ~13,318 LOW. `CAM-004` flags `In Progress` (31,280 rows) — a stakeholder decision, not a defect.

Part B — What We Will Do (POC target)

Add the two missing stages and make the whole chain incremental + checkpointed in Azure:

```
Salesforce Campaign
  -> [1 INGEST]   raw.salesforce_campaign         (append-only, duplicates OK)
  -> [2 STAGE]    staging.campaign_latest         (1 row per Id, no duplicates)
  -> [3 CHECK]    dq.* rules over staging          (flag violations, block criticals)
  -> [4 CLEAN]    clean.campaign                  (normalized, only rows that pass gate)
  -> [5 PUSH-READY] writeback.campaign_pending    (approved corrections queued for Salesforce)
```

Each arrow writes a checkpoint into a control/state table so any run can resume and any auditor can re-derive "where are we" from the start.

Part C — Stage By Stage

Stage 1 — Ingest (raw, duplicates allowed)

- Runs `python.py` for Campaign in one of three modes chosen automatically:
  - **Full** — first load, table empty.
  - **Incremental** — `WHERE SystemModstamp > last_watermark AND <= now`.
  - **ResumeMissing** — last run not `Succeeded`; loads only `Id NOT IN ctl.loaded_salesforce_campaign_ids`.
- Raw is **append-only**. We never dedupe here. Re-runs can legitimately create duplicate `Id`s (same record modified twice, or an overlap window). That is fine — this is history.

Checkpoint written: a row in `ctl.etl_run_control` (status, rows) and the advanced `ctl.watermark_control.last_watermark_value`.

Check from the start at any time:

```sql
-- Where is ingest right now?
SELECT TOP 5 run_id, load_type, status, rows_extracted, rows_loaded, start_time, end_time
FROM ctl.etl_run_control
WHERE object_name = 'Campaign'
ORDER BY run_id DESC;

SELECT object_name, last_watermark_value, last_success_run_id
FROM ctl.watermark_control
WHERE object_name = 'Campaign';

-- Raw is allowed to have duplicates:
SELECT COUNT(*) AS raw_rows,
       COUNT(DISTINCT CONVERT(VARCHAR(18), Id)) AS distinct_ids
FROM raw.salesforce_campaign;   -- raw_rows >= distinct_ids is expected
```

Stage 2 — Stage (latest, no duplicates) — GATE

- Rebuild `staging.campaign_latest` directly from `raw.salesforce_campaign` via `EXEC staging.refresh_campaign_latest;` (procedure in `database/staging/campaign_latest_table.sql`).
- Dedup rule: keep rank 1 per `Id` ordered by `SystemModstamp DESC`; exclude soft-deleted rows.
- **Gate 2 (must pass):** staging has exactly one row per `Id` and zero duplicates.

```
-- GATE 2: staging must be unique per Id. This must return 0 rows.
SELECT CONVERT(VARCHAR(18), Id) AS Id, COUNT(*) AS n
FROM staging.campaign_latest
GROUP BY CONVERT(VARCHAR(18), Id)
HAVING COUNT(*) > 1;

-- Evidence of dedup: how many raw duplicates were collapsed
SELECT
  (SELECT COUNT(*) FROM raw.salesforce_campaign
     WHERE COALESCE(LOWER(LTRIM(RTRIM(CONVERT(NVARCHAR(20),IsDeleted)))),'false')
           NOT IN ('true','1','yes','y')) AS raw_non_deleted,
  (SELECT COUNT(*) FROM staging.campaign_latest) AS staging_rows;
```

If Gate 2 fails, **stop**. Do not proceed to checks or clean.

Stage 3 — Check (data quality) — GATE

- Apply the 19 Campaign rules over `staging.campaign_latest`, writing violations to the `dq` layer (`dq.dq_exceptions` / the campaign exceptions table), tagged by rule id and severity.
- Incremental: the DQ runner only scans rows with `SystemModstamp > dq_cursor`, ordered by watermark, in bounded batches, then advances the cursor. A full re-scan is available by resetting the cursor to NULL.
- **Gate 3 (must pass to start clean/final): 0 CRITICAL** failures for rows in scope. HIGH/MEDIUM/LOW are recorded but do not block — they route to review.

```
-- GATE 3: no critical failures allowed before Clean/Final may start.
-- severity lives on dq.dq_rule_catalog; dq.dq_exceptions tracks resolution_status.
SELECT r.severity, COUNT(*) AS violations, COUNT(DISTINCT e.record_id) AS affected_campaigns
FROM dq.dq_exceptions e
JOIN dq.dq_rule_catalog r ON r.rule_id = e.rule_id
WHERE e.object_name = 'Campaign' AND e.resolution_status = 'Open'
GROUP BY r.severity
ORDER BY CASE r.severity WHEN 'CRITICAL' THEN 1 WHEN 'HIGH' THEN 2
                         WHEN 'MEDIUM' THEN 3 ELSE 4 END;

-- Must return 0 for Final to begin:
SELECT COUNT(*) AS critical_open
FROM dq.dq_exceptions e
JOIN dq.dq_rule_catalog r ON r.rule_id = e.rule_id
WHERE e.object_name = 'Campaign' AND r.severity = 'CRITICAL' AND e.resolution_status = 'Open';
```

Where is the DQ runner? Inspect the rule execution state (watermark cursor per rule):

```
SELECT r.rule_id, r.check_name, s.last_source_watermark_value,
       s.last_run_status, s.last_run_rows_checked, s.last_run_failed_count
FROM dq.dq_rule_catalog r
JOIN dq.rule_execution_state s ON s.rule_id = r.rule_id
WHERE r.object_name = 'Campaign'
ORDER BY r.rule_id;
```

Stage 4 — Clean (normalized, gated build)

> **In plain English (for the business):** the "clean" step takes the deduplicated campaigns and **tidies four fields automatically** — currency codes to a standard 3-letter uppercase form, campaign **status** spelling/spacing, the **active** flag to a simple Yes/No, and the campaign **year** filled in from its start date. It **never deletes or overwrites blindly**: the original

> value is kept next to the tidied one, and anything the computer is unsure about is **flagged for a person to review** rather than changed. It only runs when there are **no critical problems** left, so we never "clean" bad data.

> **Demonstrated result (2026-08-05, Azure POC):** 40,847 campaigns -> `clean.campaign`, **0 flagged for review**. Active flag standardized to Yes/No on all 40,847 (a format change from Salesforce text `true`/`false`, not 40k errors); campaign **Year filled/corrected on 17,735 (~43%)** from StartDate; **currency 0 changes** and **status 0 changes** (already consistent). No critical data was touched — the build ran only because 0 critical issues remained. **25 campaigns** where `AmountWon > AmountAll` were raised into `dq.alert` (CAM-008, `cleaned = 0`) for finance to review — their figures were left unchanged.

- Lives in its **own `clean` schema** (separate from `staging`) and is built by the procedure `clean.refresh_campaign` (file `database/clean/campaign_table.sql`) into the table `clean.campaign`.
- Build **only if Gate 3 passed** (0 critical). The procedure enforces this itself: it counts open CRITICAL Campaign exceptions and `RAISERROR`s (refuses to build) if any exist.
- "Clean" here is intentionally simple and safe (no business overwrites without approval). It **corrects 4 checks** and keeps both the original and cleaned value side by side:
  - **CAM-006** `CurrencyIsoCode` -> trim + UPPER (`CurrencyIsoCode_clean`),
  - **CAM-004** `Status` -> trim + collapse spaces + canonical case, and a **NULL/blank status is defaulted to `Aborted`** (`Status_clean`),
  - **CAM-017** `IsActive` boolean text -> `1/0` (`IsActive_clean`),
  - **CAM-015** `Year__c` -> derived `YEAR(StartDate)` (`Year_clean`),
  - plus typed `StartDate_clean` / `EndDate_clean` via `TRY_CONVERT` (invalid -> NULL).
- Keys/lineage carried unchanged: `Id`, `SystemModstamp` (ETL lineage columns are omitted from clean because their names differ between staging builds; `SystemModstamp` is the stable lineage key).
- Rows that fail non-critical rules are **never dropped** — they are kept and **tagged** (`clean_flag = 'REVIEW'`, `review_reason` lists the reasons: CAM-003 name, CAM-005 start>end, invalid dates, unrecognized status, non-boolean IsActive, invalid currency).
- **Business alerts (`dq.alert`):** issues the clean step **cannot** safely auto-fix are written to a simple, business-facing table `dq.alert` (file `database/dq/dq_alert_table.sql`) — like `dq.dq_exceptions`, but with a **`cleaned` flag** (`1` = the pipeline already fixed it, informational; `0` = a person still needs to act). For now only **CAM-008 (AmountWon > AmountAll)** is raised there with `cleaned = 0`, because a "won exceeds total" figure is a real discrepancy that needs human review, not an automatic edit. On the current data this flags **25 campaigns**; the alert `current_value` records `Won=… ; All=… ; Excess=…`.
- **Final cannot start until this gate holds:** Clean is only populated for the in-scope, non-critical set.

```
-- Final gate check before/after building clean:
-- 1) unique per Id (inherited from staging)
SELECT CONVERT(VARCHAR(18), Id) AS Id, COUNT(*) n
FROM clean.campaign GROUP BY CONVERT(VARCHAR(18), Id) HAVING COUNT(*) > 1;  -- expect 0

-- 2) no critical-failing Id leaked into clean
SELECT COUNT(*) AS bad
FROM clean.campaign c
WHERE EXISTS (SELECT 1 FROM dq.dq_exceptions e
              JOIN dq.dq_rule_catalog r ON r.rule_id = e.rule_id
              WHERE e.object_name='Campaign' AND r.severity='CRITICAL'
                AND e.resolution_status='Open'
                AND e.record_id = CONVERT(VARCHAR(18), c.Id));  -- expect 0
```

Stage 5 — Push-ready (writeback queue, approval gated)

- Corrections that a reviewer approves are queued in `writeback.campaign_pending` with: `Id`, field, old value, new value, rule/reason, approver, status (`Pending`/`Approved`/`Pushed`).
- Nothing is written to Salesforce automatically. A separate, approved job pushes only `Approved` rows back via the Salesforce API, then marks them `Pushed` with a timestamp and result.

- This is the "ready to push into Salesforce" endpoint of the POC. Actual push is the last, guarded step.

```
-- What is ready to push, and what already went back?
SELECT status, COUNT(*) FROM writeback.campaign_pending
WHERE object_name = 'Campaign' GROUP BY status;
```

Part D — Incremental + "Check From The Start At Any Point"

Every stage has a cursor/log so the pipeline is restartable and auditable:

Stage	Cursor / checkpoint	Reset-to-start action
Ingest	`ctl.watermark_control.last_watermark_value` + `ctl.etl_run_control`	set watermark NULL -> next run is Full from the start
Ingest resume	`ctl.loaded_salesforce_campaign_ids`	drives ResumeMissing after a crash
Stage	rebuild is deterministic from raw	re-run refresh; output is idempotent
Check	`dq.rule_execution_state.last_source_watermark_value` (per rule)	set NULL -> full re-scan from the start
Clean	derived from staging + passing DQ	drop/rebuild; idempotent
Push-ready	`writeback.campaign_pending.status`	approvals are the audit trail

One combined "where are we" query answers state at any moment:

```
SELECT 'ingest'  AS stage, CAST(last_watermark_value AS VARCHAR(30)) AS cursor_value
FROM ctl.watermark_control WHERE object_name='Campaign'
UNION ALL
SELECT 'raw_rows', CAST(COUNT(*) AS VARCHAR(30)) FROM raw.salesforce_campaign
UNION ALL
SELECT 'staging_rows', CAST(COUNT(*) AS VARCHAR(30)) FROM staging.campaign_latest
UNION ALL
SELECT 'critical_open', CAST(COUNT(*) AS VARCHAR(30))
FROM dq.dq_exceptions e JOIN dq.dq_rule_catalog r ON r.rule_id = e.rule_id
WHERE e.object_name='Campaign' AND r.severity='CRITICAL' AND e.resolution_status='Open';
```

Because each stage is idempotent and cursor-driven, a run can stop anywhere and safely resume — and an auditor can reconstruct the exact position from these tables without reading logs.

Part E — The POC (Fully On Azure)

Use Azure Portal plus Cloud Shell only. Follow these steps in order and do not skip ahead.

Step 0 — Requirements

1. Subscription access to create Azure resources (`AI Infrastructure`).
2. Microsoft Entra admin rights on the SQL logical server.
3. Repository available in Cloud Shell so `sqlcmd` can run workspace files.

Step 1 — Create Azure resources in order

Do these in order. Each step's notes/details are placed directly under that step.

1. Create resource group `Quality-check-poc` in UK South.
2. Create Azure SQL logical server `quality-check-poc-sql` and database `SalesforceDW`.
3. Open `SalesforceDW` Query editor once; if prompted, click **Allowlist IP**, wait up to 5 minutes, then reconnect with Microsoft Entra authentication.
4. Provide the Salesforce secret — pick **one** option:
   - **Option A (manual, recommended for POC)** — no Key Vault; feed the secret at runtime.
   - **Option B (Key Vault)** — create `kv-quality-check-poc` + secret `salesforce-client-secret` (needs RBAC rights).

   Decision: Option A selected — manual secret; Key Vault skipped (RBAC blocked). Supply the secret at ingest time (Step 4, Stage 1).

   ##### Option A — Manual secret (no Key Vault) — recommended for POC

   Nothing to provision here. You supply the secret later, at **ingest time (Step 4, Stage 1)**, using whichever ingest path you run (ADF secure string, or `python.py` local config / env var). The committed `config.json` keeps `"client_secret": "__FROM_KEY_VAULT__"`; the real value is never committed. RBAC does not block this option.

   ##### Option B — Azure Key Vault (production-style; needs RBAC rights)

   Create the vault with these wizard values:

   - **Basics** — Subscription `AI Infrastructure`; Resource group `Quality-check-poc`; Name `kv-quality-check-poc`; Region `UK South`; Pricing tier `Standard`; Soft-delete `Enabled`; Purge protection `Disable` for POC.
   - **Access configuration** — Permission model `Azure role-based access control (RBAC)`.
   - **Networking** — Connectivity `Public endpoint`; allow public access for POC simplicity.
   - **Tags** (optional) — `Environment=POC`, `Project=Quality-check-poc`, `Owner=<your-name-or-team>`.
   - **Review + create** — verify RG `Quality-check-poc` and Region `UK South`, then `Create`.

   Add the secret (fast path):

   1. Open `kv-quality-check-poc` -> left menu **Objects** -> **Secrets** -> **+ Generate/Import**.
   2. Upload options `Manual`; Name `salesforce-client-secret`; Secret value = the real client secret.
   3. Content type (optional) `text/plain`; Activation/Expiration empty for POC; Enabled `Yes`; `Create`.

   Then wire it up:

   - Assign role **Key Vault Secrets User** to managed identity `quality-check-poc-adf`.
   - Update/test the ADF linked service Key Vault reference.

   Checkpoint (Option B):

   - Resource `kv-quality-check-poc` exists in RG `Quality-check-poc`.
   - Secret `salesforce-client-secret` exists.
   - IAM: `Key Vault Secrets User` -> `quality-check-poc-adf`.
   - ADF linked service test: `Succeeded`.

   Troubleshooting (Option B):

   - `Secrets` **not in left menu:** confirm you opened **Key vaults** (not **Managed HSM**); if the menu shows only favorites, press **Ctrl+Shift+F**, pin **Secrets**; otherwise open **Objects -> Secrets**.
   - **Direct link:** `https://portal.azure.com/#resource/subscriptions/a5bd9c81-c288-4a80-9e39-bb06f8d35b5f/resourceGroups/Quality-check-poc/providers/Microsoft.KeyVault/vaults/kv-quality-check-poc/secrets`

- `The operation is not allowed by RBAC`**:** you lack a data-plane role. Get **Key Vault Secrets Officer** on the vault (ask an admin if **Add role assignment** is disabled), wait 2-5 minutes, retry.

Security rule (both options): never store the real secret in committed repo files or markdown. Keep a placeholder (e.g. `__FROM_KEY_VAULT__`) in committed config and resolve at runtime.

5. Create storage account `qualitycheckpocsa` with Hierarchical namespace ON, then create container `raw`.

   Storage account wizard picks (use these exact values):

   - **Basics**
     - Subscription: `AI Infrastructure`
     - Resource group: `Quality-check-poc`
     - Storage account name: `qualitycheckpocsa`
     - Region: `UK South`
     - Primary service: `Azure Blob Storage or Azure Data Lake Storage Gen2`
     - Performance: `Standard` (general-purpose v2)
     - Redundancy: `LRS` (Locally-redundant storage) — cheapest, fine for POC
   - **Advanced**
     - Hierarchical namespace: **Enabled** (REQUIRED — makes it ADLS Gen2). The portal default is **Disabled**, so you must turn it ON in the **Advanced** tab. It **cannot** be changed after creation — if created Disabled, delete and recreate.
     - Leave other options at defaults for POC
   - **Networking / Data protection / Encryption**: defaults are fine for POC
   - **Review + create**: on the review screen, confirm **Enable hierarchical namespace = Enabled**, verify RG `Quality-check-poc` and Region `UK South`, then `Create`

   After the account is created:

   - Open the account -> **Containers** -> **+ Container** -> Name `raw` -> Create.

   Status: **Done** — storage account `qualitycheckpocsa` created (hierarchical namespace `Enabled`; Standard / LRS / Hot). Next: create the `raw` container.

6. Create Data Factory `quality-check-poc-adf`.

   Data Factory wizard picks (use these exact values):

   - **Basics**
     - Subscription: `AI Infrastructure`
     - Resource group: `Quality-check-poc`
     - Name: `quality-check-poc-adf`
     - Region: `UK South` (the wizard may default to `East US` — change it)
     - Version: `V2` (ignore the "Fabric Data Factory" upsell for this POC)
   - **Git configuration**: select **Configure Git later** (keep POC simple; link Git afterward if needed)
   - **Networking / Advanced / Tags**: defaults are fine for POC
   - **Review + create**: verify RG `Quality-check-poc` and Region `UK South`, then `Create`

   Status: review values confirmed — `quality-check-poc-adf`, RG `Quality-check-poc`, Region `UK South`, `V2`, Public endpoint. Safe to `Create`.

7. In Data Factory -> Identity, confirm System assigned = On and copy Object (principal) ID.

   Status: **Done** — ADF Object (principal) ID: `35527fc6-2247-4355-9011-996c291fef95` (use this as `ADF_OBJECT_ID` in Step 3).

8. Grant IAM to the Data Factory managed identity:

   - Storage: **Storage Blob Data Contributor** (on `qualitycheckpocsa`).

- Key Vault: **not needed** — Option A selected (no Key Vault).

If **Add role assignment** is disabled (you lack Owner / User Access Administrator), use the **account-key workaround** instead of managed-identity RBAC — no admin needed:

1. Open `quality-check-poc-adf` in the Portal -> click **Launch studio** (opens `adf.azure.com` in a new tab). **Manage** and **Linked services** live inside the Studio, not the Portal page.
2. In the Studio's far-left icon rail, click **Manage** (toolbox icon) -> **Connections** -> **Linked services** -> **+ New** -> **Azure Data Lake Storage Gen2**.
3. Fill the **New linked service** form:
   - Name: `ls_adls_qualitycheckpocsa`
   - Connect via integration runtime: `AutoResolveIntegrationRuntime`
   - Authentication type: `Account key`
   - Account selection method: `From Azure subscription` (ADF pulls the key automatically — no need to copy it manually)
   - Azure subscription: `AI Infrastructure`
   - Storage account name: `qualitycheckpocsa`
4. **Test connection** (`To linked service`) -> **Create**. Use this linked service for the `raw` container.

Fallback — if `From Azure subscription` fails, use `Enter manually`:

- Open `qualitycheckpocsa` -> **Security + networking** -> **Access keys** -> **Show** -> copy the **key1 connection string**, then paste it into the linked service and Test again.

Notes:

- The account key is a broad secret — keep it only in ADF (secure string), never in the repo.
- Preferred long-term: ask an admin to assign **Storage Blob Data Contributor** to `quality-check-poc-adf`, then switch the linked service to **Managed identity**.
- A scoped **SAS token** is a middle-ground alternative to the full account key.

Stop here and verify all resources exist before Step 2.

Verification checklist (all must pass before Step 2):

1. Resource group `Quality-check-poc` lists: `quality-check-poc-sql`, `SalesforceDW`, `qualitycheckpocsa`, `quality-check-poc-adf`.
2. `SalesforceDW` -> Overview -> Status = `Online`.
3. `qualitycheckpocsa` -> **Containers** -> `raw` exists.
4. `quality-check-poc-adf` -> **Identity** -> System assigned = `On` (Object ID `35527fc6-2247-4355-9011-996c291fef95`).
5. ADF Studio -> **Manage** -> **Linked services** -> `ls_adls_qualitycheckpocsa` test = `Succeeded`.

Status: item 1 **Done** — all resources present in RG `Quality-check-poc` (a `kv-quality-check-poc` vault also exists but is unused under Option A; leave or delete later). Items 2-5 still to confirm.

If all pass, proceed to Step 2.

Step 2 — Deploy SQL objects in order

You can deploy either from **Cloud Shell** (the `>_` icon in the Portal top bar) or, if Cloud Shell is unavailable, from the **Portal Query editor**. Pick one.

Option 1 — Cloud Shell (sqlcmd):

```
cd Quality_layer_check/database
sqlcmd -S quality-check-poc-sql.database.windows.net -d SalesforceDW -G -N -C -b -i _deploy.sql
sqlcmd -S quality-check-poc-sql.database.windows.net -d SalesforceDW -G -N -C -b -i "../DQ Frame
work/DQ_framework_creation_incremental.sql"
```

```
sqlcmd -S quality-check-poc-sql.database.windows.net -d SalesforceDW -G -N -C -b -i "../DQ Frame
work/DQ_framework_seed_checks_from_existing_prod.sql"
```

If `sqlcmd -G` returns `Default account could not be found`, you are not in an authenticated Entra context. Run the commands in Azure Cloud Shell.

Option 2 — Portal Query editor (no Cloud Shell):

1. Portal -> `SalesforceDW` -> **Query editor (preview)** -> sign in with **Microsoft Entra**.
2. Open the pre-merged file `Quality_layer_check/database/deploy_merged.sql` (it already inlines every `_deploy.sql` include in order, plus the DQ framework), copy all, paste into the editor, **Run**.
3. If the editor errors on a `GO` batch or times out, run the file in a few smaller chunks (split on `GO` boundaries) until it completes.
4. Re-run is safe: the scripts are written to be idempotent.

Status: **Done** — SQL objects deployed via Portal Query editor (Option 2). Next: Step 3.

Step 3 — Create the Data Factory SQL principal

This creates `quality-check-poc-adf` inside `SalesforceDW`.

Option 1 — Cloud Shell (sqlcmd):

```
sqlcmd -S quality-check-poc-sql.database.windows.net -d SalesforceDW -G -N -C -b -v
ADF_OBJECT_ID="35527fc6-2247-4355-9011-996c291fef95" -i ctl/create_adf_sql_user.sql
```

Option 2 — Portal Query editor (no Cloud Shell): paste and Run this (GUID already inlined):

```
SET NOCOUNT ON;

DECLARE @ObjectId nvarchar(64) = N'35527fc6-2247-4355-9011-996c291fef95';

IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = N'quality-check-poc-adf')
BEGIN
    DECLARE @CreateUserSql nvarchar(max) =
        N'CREATE USER [quality-check-poc-adf] FROM EXTERNAL PROVIDER WITH OBJECT_ID = '''
        + @ObjectId + N''';';
    EXEC (@CreateUserSql);
END;

IF NOT EXISTS (
    SELECT 1 FROM sys.database_role_members drm
    JOIN sys.database_principals r ON r.principal_id = drm.role_principal_id
    JOIN sys.database_principals m ON m.principal_id = drm.member_principal_id
    WHERE r.name = N'db_owner' AND m.name = N'quality-check-poc-adf')
BEGIN
    ALTER ROLE db_owner ADD MEMBER [quality-check-poc-adf];
END;

SELECT name, type_desc FROM sys.database_principals WHERE name = N'quality-check-poc-adf';
```

name	type_desc
quality-check-poc-adf	EXTERNAL_USER

Validate:

```
SELECT name, type_desc
FROM sys.database_principals
WHERE name = 'quality-check-poc-adf';
```

If principal creation fails, wait 2-10 minutes for Entra propagation and rerun Step 3.

Status: **Done** — principal `quality-check-poc-adf` created (`EXTERNAL_USER`, `db_owner`).

> Note: `SalesforceDW` is **serverless** and auto-pauses when idle. A `Database ... is not currently available. Please retry` error just means it is resuming — wait ~30-60s and retry the query.

Step 4 — Run pipeline stages in sequence (with gates)

1. Stage 1 Ingest:
    - Run Path A (ADF Copy) first.

      Feed the Salesforce secret in the ADF Salesforce linked service:

      - **Option B (Key Vault):** set the client secret to an `Azure Key Vault` reference to secret `salesforce-client-secret`.
      - **Option A (manual):** set the client secret field type to `Secure string` and paste the value directly. ADF stores it encrypted; no Key Vault role is needed.

      Path A walkthrough (Option A manual) — do these in ADF Studio:

      1. **Salesforce source linked service:** Manage -> Linked services -> **+ New** -> search `Salesforce` -> pick **Salesforce V2** (the general object/SOQL connector; **not** "Salesforce Service Cloud V2", which is for Service Cloud).
          - Name: `ls_salesforce_poc`
          - Connect via: `AutoResolveIntegrationRuntime`
          - Environment URL: `https://humanappeal.my.salesforce.com`
          - Authentication type: `OAuth 2.0 Client Credential`
          - Client Id (Consumer Key): your Salesforce `client_id`
          - Client secret (Consumer Secret): choose `Secure string` and paste the real secret
          - API version: `67.0` (optional)
          - **Test connection** -> **Create**
      2. **SQL sink linked service** (uses the Step 3 managed-identity principal): Manage -> Linked services -> **+ New** -> **Azure SQL Database**.
          - Name: `ls_sql_salesforcedw_POC`
          - Account selection: `From Azure subscription` -> server `quality-check-poc-sql` -> database `SalesforceDW`
          - Authentication type: `System-assigned managed identity`
          - **Test connection** -> **Create** (works because `quality-check-poc-adf` has `db_owner`)
      3. **Build the pipeline `pl_ingest_campaign`.**

          What you are building: a pipeline of **three activities that run in sequence**, so every ingest is logged and auditable:

          ```
          StartRun (Lookup)  --success-->  CopyCampaign (Copy data)  --success-->
          FinishRun (Script)
          open a run row            move Salesforce -> raw              close the run row
          ```

```
capture run_id            (append rows + lineage)            write final row
counts
```

Why three activities instead of just a Copy? The `StartRun` / `FinishRun` pair writes a row in `ctl.etl_run_control` **before and after** the copy, so you always have a run log (who ran, when, how many rows, success/fail). That log is what makes the pipeline restartable and auditable.

Create the empty pipeline first:

- Open **Author** — the **pencil** icon in the Studio's far-left rail (top to bottom: Home, **Author/pencil**, Monitor/gauge, Manage/toolbox). If the rail shows icons only, it is still the second icon from the top.
- Click **+** next to *Factory Resources* -> **Pipeline**, and set the name to `pl_ingest_campaign`.
- Keep the **Activities** pane open on the left; you will drag three activities onto the canvas in the next sub-steps.

3a. `StartRun` (Lookup) — open a run row and capture its `run_id`.

This activity inserts a "Running" row into the control table and returns the new `run_id`, which the later activities reuse so all three write to the *same* run.

- From the **Activities** pane -> **General**, drag a **Lookup** activity onto the canvas and name it `StartRun`.
- **Settings** tab -> **Source dataset**: this dropdown lists **datasets**, not linked services, so on a new factory it is **empty** — you must create one. A dataset is a thin object that sits on top of a linked service (the connection):

  ```
  Linked service  ls_sql_salesforcedw_POC   = the CONNECTION (server + db +
  auth)
        ▲
  Dataset         points at that linked service = what the activity selects
        ▲
  Activity        the Lookup needs a DATASET here
  ```

  - Click **+ New** next to **Source dataset** -> **Azure SQL Database** -> **Continue**.
  - **Linked service:** `ls_sql_salesforcedw_POC`. **Table name:** pick any table (e.g. `ctl.etl_run_control`) — it is ignored once you use a query below. Click **OK**.
- Back in **Settings**, select **Use query** -> **Query** -> paste:

  ```sql
  INSERT INTO ctl.etl_run_control (object_name, load_type, start_time, status)
  VALUES (N'Campaign', N'Full', SYSUTCDATETIME(), N'Running');
  SELECT CAST(SCOPE_IDENTITY() AS BIGINT) AS run_id;
  ```

- Tick **First row only** (the query returns exactly one `run_id`).
- Drag the green **success** arrow from `StartRun` to where `CopyCampaign` will sit.

3b. `CopyCampaign` (Copy data) — move Salesforce rows into `raw`.

This is the actual extract-and-load: it reads Campaign from Salesforce and appends every row to `raw.salesforce_campaign`, tagging each row with lineage so you can trace which run produced it.

- Drag a **Copy data** activity (Activities -> **Move & transform**) onto the canvas and name it `CopyCampaign`.

- **Source** tab -> **+ New** dataset -> **Salesforce** -> linked service `ls_salesforce_poc` -> Object API name `Campaign` (or supply a SOQL query such as `SELECT ... FROM Campaign`).

  > **Common mistake — Source dataset set to Azure SQL instead of Salesforce.** The Copy's **Source** must be a **Salesforce** dataset, not SQL. If the Source's **Learn more** link reads `connector-azure-sql-database` (or you see `Use query / Table / Stored procedure` SQL options), you reused/created the wrong dataset. **Import schemas** on the **Mapping** tab will then fail or map nothing. Fix: **+ New** dataset -> **Salesforce** -> `ls_salesforce_poc` -> object `Campaign`, and confirm the Source's Learn more link now says `connector-salesforce`. Only the **Sink** stays Azure SQL Database (`raw.salesforce_campaign`).

- **Source** tab -> **Additional columns** -> add these 3 lineage values (correct ETL practice — do **not** leave them NULL):

Column	Value
`_etl_run_id`	`@activity('StartRun').output.firstRow.run_id`
`_etl_extracted_at_utc`	`@utcnow()`
`_etl_source_object`	`Campaign` (static text)

- **Sink** tab -> **+ New** dataset -> **Azure SQL Database** -> linked service `ls_sql_salesforcedw_POC` -> table `raw.salesforce_campaign`.

- **Mapping** tab -> **Import schemas** -> confirm the Salesforce fields map to the `raw` columns (names match, all target columns are `NVARCHAR(MAX)`) and the 3 additional columns map to `_etl_run_id` / `_etl_extracted_at_utc` / `_etl_source_object`.

  > **If Import schemas shows only the "Additional expressions" (not the Salesforce fields):** the connector could not fetch the object schema. Easiest fix for a raw load — **Clear** the mapping and leave it **empty**. Because every `raw.salesforce_campaign` column is `NVARCHAR(MAX)` and the names already match Salesforce field names, ADF **auto-maps by name at runtime**, including the 3 `_etl_*` additional columns. (A `Value Required` warning means an Additional column on the **Source** tab lost its value — re-enter the three.) To map explicitly instead, click **Preview data** on the Source first; if it returns rows, Import schemas will populate. If Preview fails, set the dataset **Object API name =** `Campaign` or switch the Source to a **SOQL Query** (`SELECT Id, Name, ... FROM Campaign`).

  > **If Import schemas pops "Please provide actual value of the parameters to get schema":** it is trying to *evaluate* the `_etl_run_id` additional column (`@activity('StartRun').output.firstRow.run_id`) at design time, but that value only exists **at runtime** after `StartRun` executes. Do **not** enter a fake value — **Cancel** the dialog, **Clear** the Mapping, and leave it empty (runtime auto-map by name resolves the expression correctly). If you must import explicitly, temporarily delete the `_etl_run_id` additional column, run Import schemas, then re-add `_etl_run_id` on the Source tab afterward.

- Drag the green **success** arrow from `CopyCampaign` to where `FinishRun` will sit.

3c. `FinishRun` (Script) — close the run row with final counts.

This updates the same run row to `Succeeded` and records how many rows the copy moved, closing the audit loop opened by `StartRun`.

- Drag a **Script** activity onto the canvas, name it `FinishRun`, and connect `CopyCampaign` --success--> `FinishRun`.

- Linked service: `ls_sql_salesforcedw_POC`. Paste into **Query**:

```
UPDATE ctl.etl_run_control
SET status = N'Succeeded',
    end_time = SYSUTCDATETIME(),
    rows_extracted = @{activity('CopyCampaign').output.rowsCopied},
    rows_loaded    = @{activity('CopyCampaign').output.rowsCopied}
WHERE run_id = @{activity('StartRun').output.firstRow.run_id};
```

4. **Test the pipeline:** click **Validate** (top toolbar) to catch config errors, then **Debug** to run it once without publishing. Watch the **Output** pane until all three activities show **Succeeded**.
5. **Publish and run for real:** when Debug succeeds, click **Publish all** to save the pipeline, then **Add trigger -> Trigger now** to execute it on demand.

- Optional: run Path B (`python.py` container job) only if you need ResumeMissing proof.

> **Path A vs Path B — which to use.** Use **Path A (ADF Copy)** as the primary loader: it is the cheapest (pure pay-per-run, ~$0.05–0.10 per Campaign run, nothing idle), needs no code or image, and is already built. Use **Path B** only to prove what Copy cannot — **Full / Incremental / ResumeMissing** mode selection and crash resume by missing `Id`.
>
> **How Path B runs on Azure (portal-triggerable, with logs):** package `python.py` as a container and run it as an **Azure Container Apps Job** — the serverless, scale-to-zero, pay-per-second model (the closest "Lambda-style" fit that still supports the native **ODBC Driver 18** the script needs). Trigger it from the portal (**Run now**) or from ADF via a Web activity; logs go to the job console / Log Analytics. Auth to Azure SQL uses the job's **managed identity** (`Authentication=ActiveDirectoryManagedIdentity`), granted a DB user exactly like the `quality-check-poc-adf` principal in Step 3. Inject the Salesforce secret at runtime via a Container Apps **secret** / env var — never bake it into the image.
>
> **Why not Azure Functions (the Lambda equivalent)?** Considered and rejected for this workload: the cheap **Consumption** plan cannot install the native **ODBC Driver 18** `pyodbc` needs and caps a run at **10 min** (a problem for the big objects later). Fixing both forces a custom container on a **Premium/Flex** plan, which costs more than the container route with no gain. **Container Apps Job wins** for this `pyodbc` ingester.

Feed the Salesforce secret for Path B:

2. Stage 2 Stage (dedup):
   - Run `EXEC staging.refresh_campaign_latest;` (rebuilds `staging.campaign_latest` directly from `raw.salesforce_campaign`, deduped to one row per Id, with the `staging_is_duplicate` / `staging_duplicate_count` / `staging_created_at` columns).
   - Gate: duplicate Id count in `staging.campaign_latest` = 0.
3. Stage 3 Check (DQ):
   - Run `EXEC dq.run_incremental_catalog_rules @ObjectNameFilter='Campaign', @MaxRowsPerRule=100000, @MaxExceptionsPerRule=500;`.
   - Gate: CRITICAL exception count = 0.
4. Stage 4 Clean:
   - Create the `clean` schema + procedure once (already in the deploy scripts): the schema is created in `00_schemas.sql` and the procedure `clean.refresh_campaign` in `database/clean/campaign_table.sql`. If your deployed DB predates these files, run the contents of `database/clean/campaign_table.sql` once in Query editor (it is a `CREATE OR ALTER PROCEDURE`, safe to re-run).
   - Build it: `EXEC clean.refresh_campaign;` (blocks itself if any CRITICAL exception is open).
   - Gate: `clean.campaign` has unique Ids and no CRITICAL leak (queries above).

- **Show the corrections (before staging vs after clean), joined on Id:**

```
-- Cleaned vs skipped summary
SELECT clean_flag, COUNT(*) AS rows
FROM clean.campaign GROUP BY clean_flag;   -- CLEAN vs REVIEW counts

-- CAM-006 currency: rows the clean step actually changed (before -> after)
SELECT TOP 20 c.Id, c.CurrencyIsoCode_raw AS before_currency, c.CurrencyIsoCode_clean
AS after_currency
FROM clean.campaign c
JOIN staging.campaign_latest s ON CONVERT(VARCHAR(18), s.Id) = CONVERT(VARCHAR(18),
c.Id)
WHERE ISNULL(c.CurrencyIsoCode_raw, N'') <> ISNULL(c.CurrencyIsoCode_clean, N'');

-- CAM-004 status: whitespace/case normalized (before -> after)
SELECT TOP 20 c.Id, c.Status_raw AS before_status, c.Status_clean AS after_status
FROM clean.campaign c
JOIN staging.campaign_latest s ON CONVERT(VARCHAR(18), s.Id) = CONVERT(VARCHAR(18),
c.Id)
WHERE ISNULL(c.Status_raw, N'') <> ISNULL(c.Status_clean, N'');

-- CAM-017 IsActive + CAM-015 Year derived from StartDate (before -> after)
SELECT TOP 20 c.Id,
       c.IsActive_raw AS before_isactive, c.IsActive_clean AS after_isactive,
       c.Year_raw     AS before_year,     c.Year_clean     AS after_year,
       c.StartDate_raw
FROM clean.campaign c
JOIN staging.campaign_latest s ON CONVERT(VARCHAR(18), s.Id) = CONVERT(VARCHAR(18),
c.Id)
WHERE ISNULL(CONVERT(NVARCHAR(20), c.IsActive_raw), N'') <>
ISNULL(CONVERT(NVARCHAR(20), c.IsActive_clean), N'')
   OR ISNULL(c.Year_raw, N'') <> ISNULL(CONVERT(NVARCHAR(20), c.Year_clean), N'');

-- What was skipped and why
SELECT TOP 20 Id, review_reason FROM clean.campaign WHERE clean_flag = N'REVIEW';
```

- **Test & explain (one business-readable summary):** run this single query to show, in one row, how many campaigns were cleaned vs need review, and how many of each field the clean step corrected:

```
SELECT
  COUNT(*)
AS total_campaigns,
  SUM(CASE WHEN clean_flag = N'CLEAN'  THEN 1 ELSE 0 END)
AS ready_no_review,
  SUM(CASE WHEN clean_flag = N'REVIEW' THEN 1 ELSE 0 END)
AS needs_review,
  SUM(CASE WHEN ISNULL(CurrencyIsoCode_raw, N'') <> ISNULL(CurrencyIsoCode_clean, N'')
THEN 1 ELSE 0 END) AS currency_fixed,
  SUM(CASE WHEN ISNULL(Status_raw, N'')          <> ISNULL(Status_clean, N'')
THEN 1 ELSE 0 END) AS status_fixed,
  SUM(CASE WHEN ISNULL(CONVERT(NVARCHAR(20), IsActive_raw), N'') <>
ISNULL(CONVERT(NVARCHAR(20), IsActive_clean), N'') THEN 1 ELSE 0 END) AS
isactive_fixed,
  SUM(CASE WHEN ISNULL(Year_raw, N'')            <> ISNULL(CONVERT(NVARCHAR(20),
Year_clean), N'') THEN 1 ELSE 0 END) AS year_fixed
FROM clean.campaign;
```

Read it back to the business like this: "*Of **total_campaigns**, **ready_no_review** are clean and ready; **needs_review** need a human. The step corrected currency on **currency_fixed**, status on **status_fixed**, the active flag on **isactive_fixed**, and filled the year on **year_fixed** — and it changed nothing critical, because the run was blocked unless zero critical issues remained.*"

- **New behaviour — Status default + business alerts (show the difference & effect):**

```
-- 1) Status: NULL/blank now defaulted to 'Aborted' (before -> after)
SELECT TOP 20 Id, Status_raw AS before_status, Status_clean AS after_status
FROM clean.campaign
WHERE Status_raw IS NULL OR LTRIM(RTRIM(CONVERT(NVARCHAR(200), Status_raw))) = N'';

-- how many campaigns received the 'Aborted' default (the effect)
SELECT COUNT(*) AS status_defaulted_to_aborted
FROM clean.campaign
WHERE (Status_raw IS NULL OR LTRIM(RTRIM(CONVERT(NVARCHAR(200), Status_raw))) = N'')
  AND Status_clean = N'Aborted';

-- 2) dq.alert: issues a human must action (currently only CAM-008 won > all)
SELECT check_name, severity, cleaned, COUNT(*) AS alerts
FROM dq.alert WHERE object_name = 'Campaign'
GROUP BY check_name, severity, cleaned;

-- the actual campaigns flagged, with the offending numbers (Won / All / Excess)
SELECT TOP 20 record_id, issue, current_value
FROM dq.alert WHERE object_name = 'Campaign' AND check_name = 'CAM-008';

-- ALL alerts to show the business (every object/check), most campaigns first
SELECT object_name, check_name, severity,
       CASE WHEN cleaned = 1 THEN N'Auto-fixed' ELSE N'Needs a person' END AS action,
       COUNT(*) AS campaigns
FROM dq.alert
GROUP BY object_name, check_name, severity, cleaned
ORDER BY campaigns DESC;
```

Read the effect back to the business: "*We defaulted **status_defaulted_to_aborted** campaigns with no status to **Aborted**, and raised **25** campaigns where the 'won' amount exceeds the 'total' amount into the alert list for finance to review — the pipeline did **not** change those numbers.*"

5. Stage 5 Push-ready:
   - Queue approved corrections in `writeback.campaign_pending`.
   - Push back to Salesforce remains a separate approved step.

Do not continue to the next stage until the gate of the current stage passes.

Step 5 — Monitor and troubleshoot

1. ADF Monitor: check pipeline and activity run status.
2. Query Editor: verify control tables (`ctl.etl_run_control`, `ctl.watermark_control`).
3. If needed, use the gate queries from Part C as the final pass/fail source.

Part F — Definition Of Done (Campaign POC)

- Raw contains history/duplicates; distinct-id < row-count is accepted.
- **Both ingest paths tested:** Path A (ADF Copy) and Path B (container `python.py`) each land into the same `raw.salesforce_campaign` with equal counts; Path B also proves Full / Incremental / ResumeMissing.
- `staging.campaign_latest` has **0 duplicate Ids** and excludes deleted rows.
- DQ runs incrementally and reports **0 CRITICAL** for the in-scope set.

- `clean.campaign` exists, is unique per `Id`, and contains **no** critical-failing record.
- Clean **actually corrects 4 checks** (CAM-006 currency, CAM-004 status case/whitespace, IsActive boolean, CAM-015 Year-from-StartDate) and **skips the rest with a recorded reason** — the cleaned/skipped counts are visible in the demonstration query.
- The pipeline can be stopped and resumed at any stage, and state is provable from control tables.

Out of scope for this POC (needs their support): the actual **write-back / push to Salesforce is NOT required** to call this POC done. Corrections stay queued in `writeback.campaign_pending` (Stage 5); the push-back is a later, approval-gated step that needs **Salesforce admin support** (integration user + write access on the Connected App) and **business sign-off**.

Once Campaign passes, repeat the same five stages for the other 8 objects (largest last), reusing the Azure services chosen in azure_migration_plan.md.

Part G — Optional Cost Estimate (3-Day POC)

Scenario: **Campaign only** (~41K rows / ~50 MB), UK South, over **3 days** — one **full load**, a few **incrementals** per day, running the **rules**, and building the **clean** table. This is a relative estimate, **not a quote** — confirm with the Azure Pricing Calculator.

Service	What drives cost	3-day estimate
Azure SQL Database (serverless, GP 1 vCore, auto-pause 1h)	vCore-seconds **while active** + ~5 GB storage	~£4–6 (near £0 while paused)
Azure Data Factory	activity-run count + Copy DIU-hours (tiny dataset)	~£1–2
ADLS Gen2	50 MB + a few thousand operations	< £0.10
Key Vault	per-secret operation	< £0.05
Log Analytics (if enabled for logs)	ingested log volume (small)	< £0.20
Total		**~£6–10 (~$8–13)**

Keep it low: leave SQL **auto-pause** on (idles to near-zero), run pipelines **on demand** (not a tight schedule), and **delete the resource group** when done. Serverless compute dominates the bill; everything else is pennies at this size. Cost scales with the *big* objects later (Opportunity, Item Allocation) — not with Campaign.## Part G — Optional Cost Estimate (3-Day POC)

Scenario: **Campaign only** (~41K rows / ~50 MB), over **3 days** — one **full load**, a few **incrementals** per day, running the **rules**, and building the **clean** table. Relative estimate in **USD**, **not a quote** — confirm with the Azure Pricing Calculator.

Service	Why it costs (billing driver)	3-day estimate (USD)
Azure SQL Database (serverless, GP 1 vCore, auto-pause 1h)	billed per **vCore-second while active** + ~5 GB storage; pauses to ~$0 when idle	~$5–8
Azure Data Factory	per **activity run** + **Copy DIU-hour** (tiny dataset)	~$1.50–3
ADLS Gen2	50 MB storage + a few thousand operations	< $0.15
Key Vault	per-secret **operation**	< $0.05
Log Analytics (if logs enabled)	per **GB ingested** (small)	< $0.25
Total		**~$8–13**

Ingest path cost comparison (Path A vs Path B vs Functions)

Both ingest paths write to the **same** `raw.salesforce_campaign`; only the *runner* differs.

Path	Per Campaign run	Fixed / idle cost	Verdict
A — ADF Copy	~$0.05–0.10 (4 DIU × few min + activity run)	**$0** (pay-per-run)	**Primary** — cheapest, no code, already built
B — Container Apps Job	**<$0.01** (usually inside the free monthly grant → ~$0)	**ACR ~$5/mo** (Basic) — or **$0** via a public Docker Hub repo	Use only to prove Full/Incremental/**ResumeMissing**
Azure Functions	~$0 on Consumption free grant	Premium plan (needed for ODBC + >10 min) **> ACR cost**	**Not used** — native ODBC + 10-min cap force a pricier container-on-Premium setup

Decision: route routine ingest through **Path A**; spin up **Path B** briefly only for the resume proof, then **delete its ACR** (and Container Apps env) to drop the fixed cost. The serverless SQL DB (on auto-pause) is shared by both paths and dominates the bill regardless of runner.

Why so low: serverless SQL auto-pauses (near-$0 idle) and Campaign is tiny, so vCore-seconds and DIU-hours are minimal.
Pricing references: Azure SQL Database serverless · Data Factory pipelines · ADLS Gen2 / Blob · Key Vault · Azure Monitor / Log Analytics.